

Link Prediction on a Multi-Relational Knowledge Graph: FB15k-237, Matched KGE, and Planned Hard-Negative & Slice Analysis

Thalia Delhaise, Lenny Briclet, André Fonseca, Rafael Gufler (DL + KG joint project)

RC2526 — Deliverable 2 (Report part 1)

1. Introduction and motivation

A **knowledge graph** (KG) is a set of binary facts (h, r, t) over a finite set of **entities** $\mathcal{E}$ and **relation types** $\mathcal{R}$. **Link prediction** (knowledge base completion) asks which missing triples are true: for a query with one side hidden, e.g. $(h, r, ?)$, rank all candidate tail entities. This is a standard **learning on graphs** task with applications to semantic query answering and maintenance of large knowledge bases.

In this project we (i) **materialize** a benchmark KG from the **FB15k-237** public triple files, (ii) run **TransE** and **ROTATE** under matched conditions, and (iii) develop **custom hard-negative** training and **relation-wise** evaluation—the main methodological extensions. A **relational GNN** is a **optional** add-on for the final phase if time allows; it is not required for the core claim.

Our **contribution** is: reproducible KGE baselines; **own PyTorch** code for harder negatives and ablations vs. defaults; and **analysis** (slices, error cases) on the **same filtered** metrics (Hits@ k , MRR).

2. Problem statement

Formal graph. $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ with $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$. The graph is **directed** and **multi-relational**: the same node pair can appear in several relations. Training sees only $\mathcal{T}_{\text{train}}$. Validation and test are disjoint held-out **positive** triples.

Data. FB15k-237 (Freebase-derived) [4]: 14,541 entities, 237 relations, 272,115 / 17,535 / 20,466 training, validation, and test triples. The benchmark is widely used; inverse relations are removed to reduce trivial inferences. We load the official split via **PyKEEN** [1]; optional RDF/edge export for a KG visualization slide is a minor technical step.

Task. Link prediction: for each test triple, rank the correct head or tail under the **filtered** setting [2] (candidates that complete a *known* triple in train, val, or test are not errors).

Success criteria. (1) Strong **aggregate** filtered MRR and Hits@1,3,10. (2) Ablations on **negative** strategies. (3) Deeper **relation-wise** and **degree** analysis and short error study. (4) **Optional:** a GNN baseline.

3. Methodology

Phase A — Classical KGE (Milestone 1–2). We implement **TRANSE** and **ROTATE** [4] with **PyKEEN** for a fair **first** line: same splits, d , optimiser, training budget, and **filtered** evaluation. The scoring function is learned; training uses margin/ranking loss and negative sampling.

Phase B — Custom negative mining (in-house PyTorch). In parallel with KGE, we will implement a **learnt** or **scheduled hard-negative** mechanism: e.g. a small g_ϕ over a subset of corruptions, or periodic **mining** in embedding space, with ablations **against** uniform and Bernoulli **defaults**.

Phase C (optional) — Relational GNN. If time allows, a shallow R-GCN (or **COMPGCN**) in **PyG/DGL** [3] on the **same** task and splits; not required for the main thesis (hard negatives + KGE + analysis).

Evaluation. In addition to global MRR and Hits@ k , we will report **slices** (relation frequency, entity degree) and a short **error** / case study.

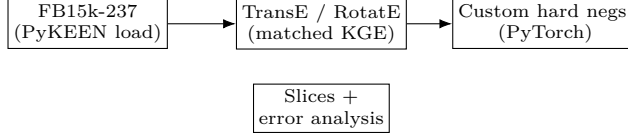


Figure 1: Core pipeline: same filtered evaluation; custom negatives and analysis on the KGE line.

4. Planned experiments and preliminary results

Planned. (1) TransE/RotatE; **match** d and budget. (2) Custom negative ablations. (3) Slice + error analysis. (4) Optional GNN. (5) Time/hardware.

Preliminary (Milestone 1). We use a custom training script (`code/train_baseline_kge.py`), running on CUDA: $d = 128$, batch 1024, Adam lr 10^{-3} , up to 15 epochs with early stopping (patience 3) on validation MRR. PyKEEN reports **realistic** filtered ranking, head and tail prediction averaged.

Table 1: Preliminary filtered metrics on the test set (realistic ranking, **both** averaged; 15 epochs max, early stopping with patience 3 on val MRR; $d = 128$, Adam, lr= 10^{-3} , seed=42).

Model	MRR	H@1	H@3	H@10	Notes
TransE	0.128	0.055	0.151	0.264	15 ep. max, bs=1024, CUDA
RotatE	0.234	0.159	0.263	0.378	15 ep. max, bs=1024, CUDA

RotatE consistently outperforms TransE across all metrics at matched hyperparameters: MRR improves from 0.128 to 0.234 (+83%), and H@10 from 0.264 to 0.378. Both models were still improving at epoch 15 (no early stopping triggered), suggesting further gains with longer training in Milestone 2. The asymmetry between head prediction (RotatE H@10=0.261) and tail prediction (RotatE H@10=0.494) motivates the planned relation-type slice analysis.

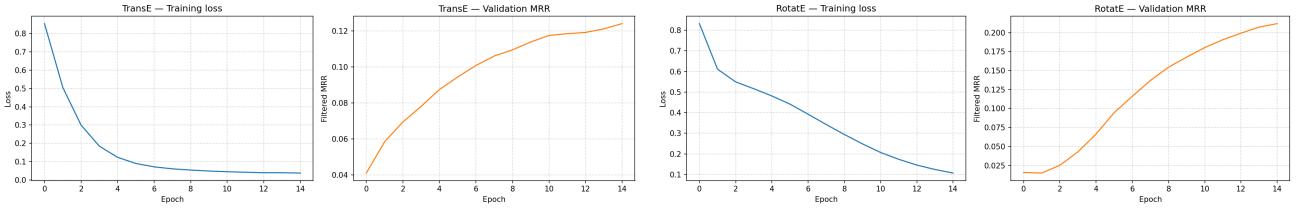


Figure 2: Training loss and validation MRR vs. epoch for TransE (left, CUDA) and RotatE (right, CUDA). Both models continue to improve at epoch 15, motivating longer runs in Milestone 2.

References

- [1] Mehdi Ali, Max Berrendorf, Charles Tapley Hoyt, Laurent Vermue, et al. PyKEEN 1.0: A python library for training and evaluating knowledge graph embeddings. *JMLR*, 22(82):1–6, 2021.
- [2] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013.
- [3] Michael Schlichtkrull, Thomas N Kipf, et al. Modeling relational data with graph convolutional networks. In *ESWC*, 2018.
- [4] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.